

计算机设计与实践

单周期CPU设计

2026 · 夏

哈工大



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

目录



课程介绍

指令系统概要

CPU的基本原理与结构

单周期CPU设计

课程基本信息

- 开课学期：大二夏季 (2026夏)
- 总学时：48学时 = 4学时理论 + 44学时实验
- 课程学分：3.0
- 项目名称：支持miniRV/miniLA指令集的SoC设计



课程目标

1. **综合**运用《数字逻辑设计》、《计算机组成原理》等基础课程知识，系统掌握计算机硬件系统设计与实现的**工程**方法
2. 锻炼对计算机硬件系统的**分析**、**设计**和**创新**能力
3. 通过计算机部件、CPU，再到SoC的逐步实现，了解一个工程项目的开发过程、锻炼**解决复杂工程**问题的能力
4. 掌握HDL、EDA工具的使用，锻炼计算机硬件系统的**开发**、**调试**能力
5. 培养团队合作意识，锻炼团队协作能力

课程安排

□ 总共48学时 = 4学时理论 + 44学时实验

□ 完成项目：支持miniRV/miniLA指令集的SoC设计

		序号	教学内容	学时
实验1： 单周期CPU 设计	{	1-1	单周期CPU与SoC设计要点	2
		1-2	单周期CPU demo工程仿真分析	2
		1-3	单周期CPU与SoC设计与实现	16
实验2： 流水线CPU 及SoC设计	{	2-1	流水线CPU及SoC设计要点	2
		2-2	流水线CPU、SoC设计与实现	22
		2-3	调试、验收、展示&答辩(面向优秀作品, 有加分)	4

设计目标

- 设计实现包含CPU、Cache、总线与外设的SoC，能够下板运行CoreMark或LLAMA2推理程序

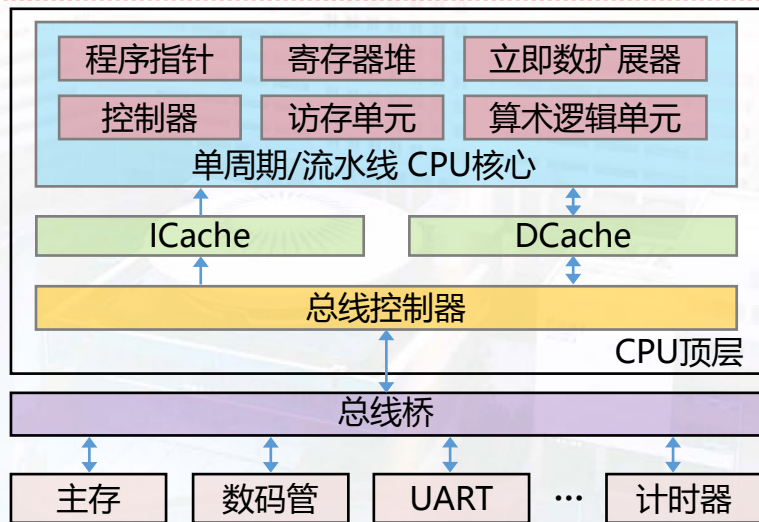
课程主要设计内容

应用程序

Bare Metal应用程序 (CoreMark、LLAMA2等)

miniRV/miniLA汇编器、编译器、IDE

SoC系统



项目内容

□ 课程采取**2人小组**合作完成：

➤ **分组方式**：自行搭配、随机分组，非特殊原因不得跨课表组队

➤ **实验1**：

- ① 小组成员分别基于A、B组指令，设计单周期CPU
- ② 分别使用Trace测试框架，测试和验证单周期CPU
- ③ 通过交流讨论，整合出支持miniRV/miniLA所有指令的单周期CPU

➤ **实验2**：

- ① 组员甲设计流水线CPU，组员乙设计SoC并完成下板验证
- ② 通过交流讨论，整合出流水线SoC，使用Trace测试框架完成调试
- ③ 共同下板测试流水线SoC，优化性能

评分: 课后作业及考勤(17%) + 系统实现(63%) + 代码及报告(20%)

一、基础分

及格: 单周期CPU实现所有指令, 跑通Basic Trace;

理想流水CPU通过功能仿真测试、单周期实现AXI并跑通lw、sw的仿真测试或trace

中等: 在及格基础上, 流水线CPU跑通Basic Trace, 单周期CPU跑通AXI Trace

良好: 在中等基础上, 流水线CPU跑通AXI Trace, 单周期SoC下板跑通C_TEST测试0~2

优秀: 在良好基础上, 流水线CPU跑通AXI Trace, 流水线SoC下板跑通CoreMark

二、加分项 (分数加到系统实现部分, 上限加到70分)

1. 实现miniLA指令集, 系统实现部分+6分
2. 为SoC增加DDR控制器, 跑通LLAMA2推理程序
3. 优化SoC性能 (提高主频、优化访存及运算等), 提升CoreMark或LLAMA2性能
4. 实现更多外设 (如4x4键盘、VGA、以太网等), 编写C语言测试程序并下板演示
5. 其他具有一定**工作量及创新性**的额外工作, 视实际情况加分

课上现场验收内容

1. 正常情况

- ① 实验1：单周期CPU的完整数据通路图
- ② 实验2：SoC下板运行**CoreMark**或**LLAMA2**程序
 - 若跑不通CoreMark及LLAMA2，检查**C_TEST测试0~2**的下板

2. 及格线

- ① 实验1：单周期CPU的完整数据通路图
- ② 实验2：**理想流水线**CPU仿真、
实现AXI总线，跑通**lw、sw指令的仿真**或**AXI Trace**

➤ 现场检查时，所有组员必须到场

平台及参考资料

- 课程实验指导书: cpu-design.p.cs-lab.top
- 《计算机组成与设计: 硬件/软件接口 (RISC-V版)》
—— David Patterson, John Hennessy
- riscv/riscv-cores-list: github.com/riscv/riscv-cores-list



RISC-V Cores and SoC Overview

This document captures the status of various cores and SoCs that endeavor to implement the RISC-V specification. Note that none of these cores/SoCs have passed the in-development RISC-V compliance suite.

Please add to the list and fix inaccuracies - see our [CONTRIBUTING](#) file for details.

Cores

Name	Supplier	Links	Capability	Priv. spec	User spec	Pri Lan
Avispado	SemiDynamics	Website	RV64	1.10	RV64GC, 2.2, multicore, V-ready	Syster
Atrevido	SemiDynamics	Website	RV64	1.10	RV64GC, 2.2, multicore, V-ready	Syster
RV32EC_P2	IQonIC Works	Website	RV32	1.11	RV32E[M]C/RV32I[M]C	Syster

目录



课程介绍

指令系统概要

CPU的基本原理与结构

单周期CPU设计

miniRV指令系统

□ miniRV是32位整型RISC-V指令集（RV32I）的子集

- 指令数：44条 (8条示例 + 36)
- 指令类型：R型、I型、S型、B型、U型、J型
- 指令长度：32位定长
- 寄存器：32个32位通用寄存器
- 寻址方式：4种
 - 寄存器寻址、立即数寻址、基址寻址、PC相对寻址

miniRV指令目录

A组蓝色, B组橙色, 各18条

- 算术运算指令 (12) —— `addi`, `lui`, `add`, `sub`, `auipc`,
`mul`, `mulh`, `mulhu`, `div`, `divu`, `rem`, `remu`
- 逻辑运算指令 (6) —— `ori`, `xor`, `xori`, `or`, `and`, `andi`
- 移位运算指令 (6) —— `slli`, `sll`, `srl`, `srli`, `sra`, `srai`
- 加载及存储指令 (8) —— `lw`, `lb`, `lbu`, `lh`, `lhu`, `sw`, `sb`, `sh`
- 条件转移指令 (6) —— `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`
- 无条件转移指令 (2) —— `jal`, `jalr`
- 比较指令 (4) —— `slt`, `slti`, `sltu`, `sltiu`

miniLA指令系统

□ miniLA是32位整型LoongArch ISA精简版（LA32R）的子集

- LoongArch是我们国家自己的指令集，完全自主可控
- 指令数：45条 (8条示例 + 37)
- 指令类型 (按指令格式):
 - 3R型、2RI5型、2RI12型、1RI20型、2RI16型、I26型
- 指令长度：32位定长
- 寄存器：32个32位通用寄存器
- 寻址方式：4种

miniLA指令目录

A组蓝色19条, B组橙色18条

- ❑ 算术运算指令 (12) —— `addi.w`, `lu12i.w`, `add.w`, `sub.w`, `pcaddu12i`,
`mul.w`, `mulh.w`, `mulh.wu`, `div.w`, `div.wu`, `mod.w`, `mod.wu`
- ❑ 逻辑运算指令 (6) —— `ori`, `xor`, `xori`, `or`, `and`, `andi`
- ❑ 移位运算指令 (6) —— `slli.w`, `sll.w`, `srl.w`, `srli.w`, `sra.w`, `srai.w`
- ❑ 加载及存储指令 (8) —— `ld.w`, `ld.b`, `ld.bu`, `ld.h`, `ld.hu`, `st.w`, `st.b`, `st.h`
- ❑ 条件转移指令 (6) —— `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`
- ❑ 无条件转移指令 (3) —— `b`, `jirl`, `bl`
- ❑ 比较指令 (4) —— `slt`, `slti`, `sltu`, `sltui`

指令集对比

- 两个指令集的指令在功能上一一对应
- 从指令编码角度对比：

对比项	miniRV	miniLA
指令格式	指令格式高度规整 opcode、funct3、funct7位置和长度固定 立即数位序重叠度高 (仅U型指令的位序不同)	指令格式比较规整 opcode位置固定，长度不固定 (4种长度) 立即数位序重叠度一般 (3种位序)
寄存器用途	rs1、rs2始终作为源寄存器 rd始终作为目标寄存器	rj、rk始终作为源寄存器 rd在条件分支指令中充当源寄存器
跳转范围	条件分支：13位偏移量， $\pm 1k$ (32位指令) 直接跳转：21位偏移量， $\pm 256k$ (32位指令)	条件分支：18位偏移量， $\pm 32k$ 直接跳转：28位偏移量， $\pm 32M$

目录



课程介绍

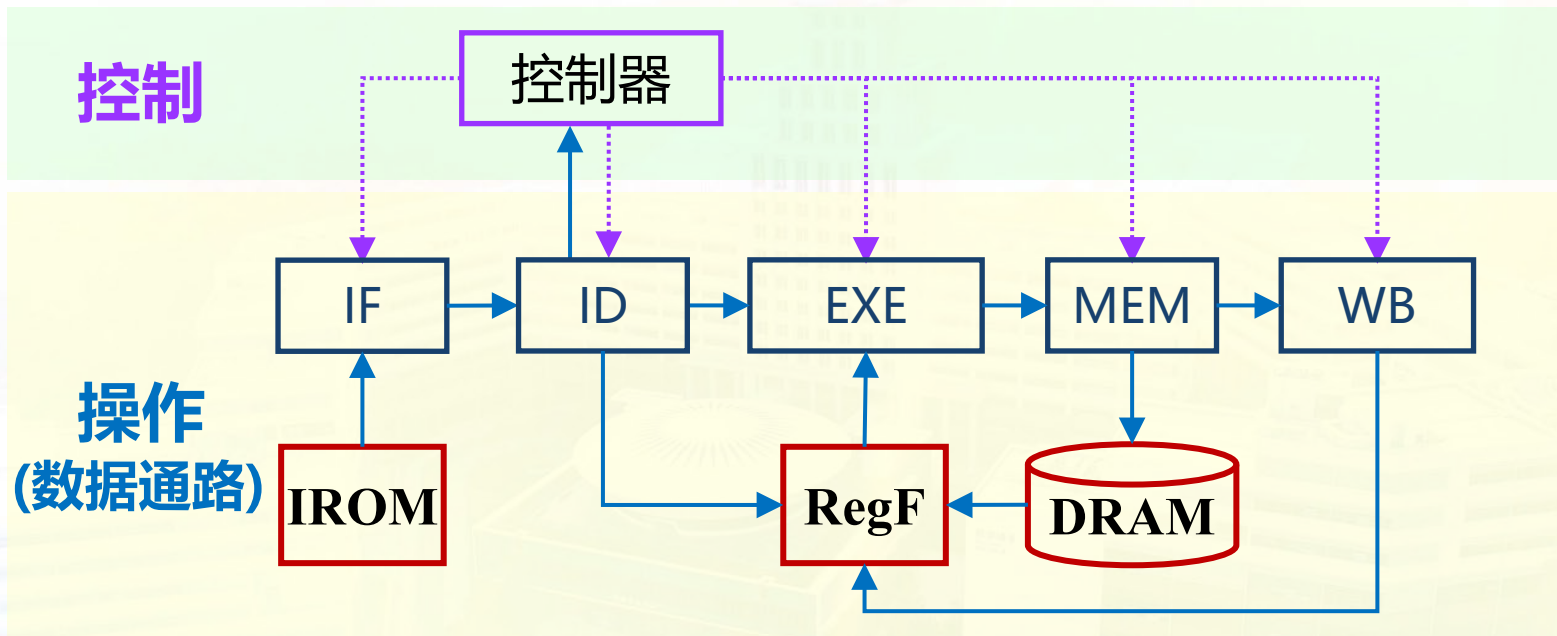
指令系统概要

CPU的基本原理与结构

单周期CPU设计

CPU执行指令的基本过程

- 指令执行的基本过程：IF-ID-EX-MEM-WB，执行受控制信号控制



CPU = 数据通路 + 控制逻辑

CPU的基本结构

□ 从结构上，CPU包含**数据通路**和**控制逻辑**

➤ 数据通路

- ◆ 数据流经路径上的所有部件构成的通路，是CPU完成数据处理的物理基础
- ◆ 包括PC、指令存储器、寄存器堆、ALU，etc

➤ 控制逻辑

- ◆ 控制运行时数据通路的具体功能，主要指控制器

目录

课程介绍

指令系统概要

CPU的功能、原理与结构

单周期CPU设计

工程化设计方法

CPU功能验证

构建指令级数据通路

分析指令功能，以基本部件为起点，构造指令级别的数据通路

综合数据通路

当所有指令的数据通路构造完毕后，将其综合为完整的数据通路

构建控制信号逻辑

确定控制信号及其取值，建立控制信号表，列出相应的逻辑表达式

实现功能部件

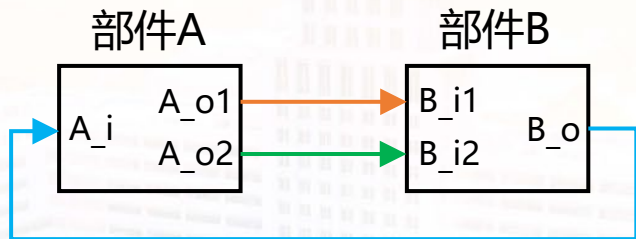
提取功能部件，确定功能及接口信号，并使用HDL描述内部具体电路行为或结构

连接和组装

根据数据通路表的连接关系，对各部件进行连接和组装

数据通路的表示

- ◆ 数据通路本质上是一个功能部件和连接关系的集合
 - ◆ 集合的元素：部件、部件的输入输出信号之间的连接关系



- ◆ 集合表示法：{ A, B, <A.A_{o1}, B.B_{i1}>, <A.A_{o2}, B.B_{i2}>, <B.B_o, A.A_i> }

- ◆ 表格表示法：

部件A	部件B	
A _i	B _{i1}	B _{i2}
B.B _o	A.A _{o1}	A.A _{o2}

———▶ 部件名称

———▶ 部件的输入信号/引脚

———▶ 由其他部件产生，并输入到当前引脚的信号

数据通路表模板

- ◆ 每条指令对应的部件和连接关系都不同
- ◆ 为了方便数据通路的综合，采用表格表示法
 - ◆ 建立数据通路表：

所属单元	取指单元				译码单元					执行单元		存储单元			
功能部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入信号	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	ram_addr	ram_wdata	din	byte_offs

第1-2行：数据通路的功能部件及其所属单元

第 3 行：各功能部件的输入信号/引脚

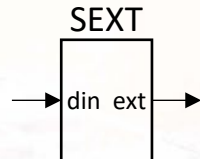
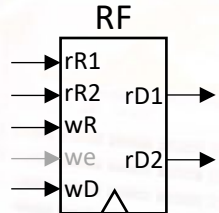
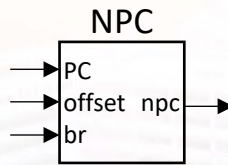
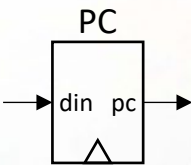
第 4+行：由其他部件产生，并连接到当前输入引脚的信号

每一行表示一条指令的数据通路

- ◆ 第2行所列部件、第3行所列接口信号均仅为参考，应根据实际需要，在设计过程中，进行相应的增删改

基本部件说明

所属单元	取指单元				译码单元					执行单元		存储单元			
功能部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入信号	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	ram_addr	ram_wdata	din	byte_offs



部件	信号	属性	释义
PC	din	输入	下一条指令地址
	pc	输出	当前指令地址

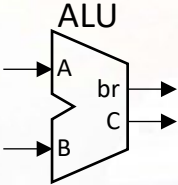
部件	信号	属性	释义
NPC	PC	输入	当前指令地址
	offset	输入	跳转的偏移量
	br	输入	条件分支是否跳转
	npc	输出	下一条指令地址

部件	信号	属性	释义
RF	rR1	输入	读地址1
	rD1	输出	读寄存器数据1
	rR2	输入	读地址2
	rD2	输出	读寄存器数据2
	wR	输入	写地址
	we	输入	写使能
	wD	输入	写寄存器数据

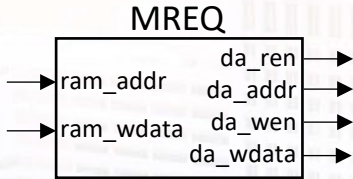
部件	信号	属性	释义
SEXT	din	输入	指令中的立即数
	ext	输出	扩展后的立即数

基本部件说明

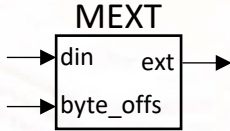
所属单元	取指单元				译码单元					执行单元		存储单元			
功能部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入信号	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	ram_addr	ram_wdata	din	byte_offs



部件	信号	属性	释义
ALU	A	输入	操作数1
	B	输入	操作数2
	C	输出	运算结果
	br	输出	标志位



部件	信号	属性	释义
MREQ	ram_addr	输入	计算得出的访存地址
	da_ren	输出	读请求信号
	da_addr	输出	读地址信号
	ram_wdata	输入	待写入主存的数据
	da_wen	输出	写请求信号
	da_wdata	输出	当前指令地址



部件	信号	属性	释义
MEXT	din	输入	主存读数据
	byte_offs	输入	访存地址的字节偏移
	ext	输出	扩展后的数据

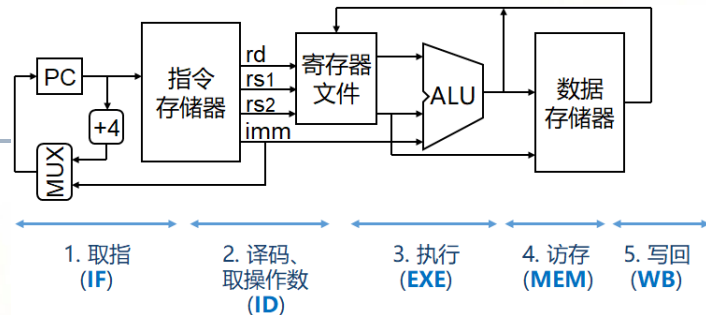
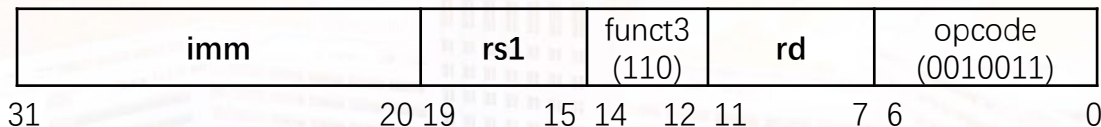
指令级数据通路构建

◆ 示例1：ori指令

◆ I-类型、逻辑或运算

◆ **ori** rd, rs1, imm $\# (rd) \leftarrow (rs1) \mid \text{sext}(\text{imm}), (PC) \leftarrow (PC) + 4$

◆ 编码格式：



单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
ori	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-

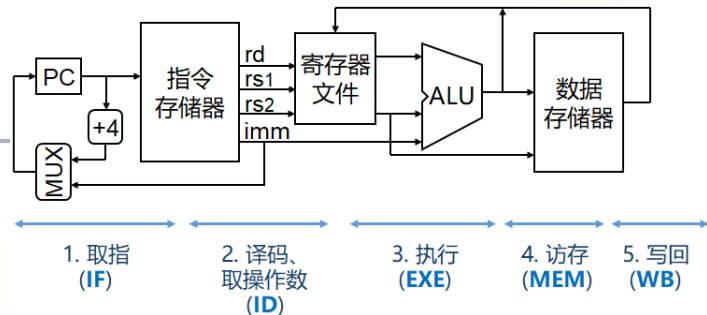
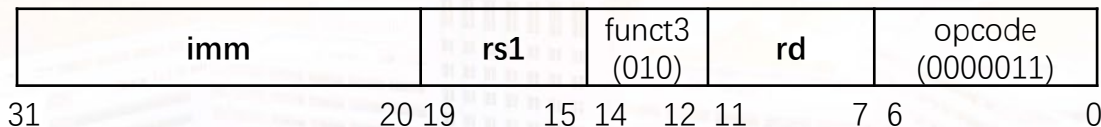
指令级数据通路构建

◆ 示例2: lw指令

◆ I-类型、读访存操作

◆ **lw** rd, offset(rs1) # $(rd) \leftarrow \text{MEM}[(rs1) + \text{sext}(\text{offset})], (PC) \leftarrow (PC) + 4$

◆ 编码格式:



单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
lw	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	MEXT.ext	IN.inst[31:20]	RF.rD1	SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]

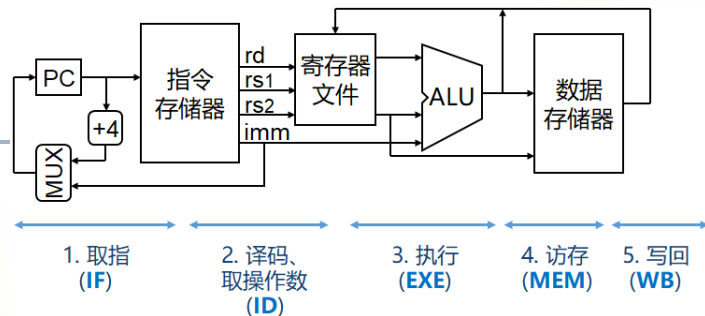
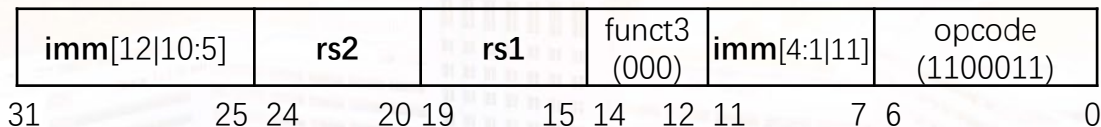
指令级数据通路构建

◆ 示例3: beq指令

◆ B-类型、分支跳转

◆ **beq** rs1, rs2, offset $\# (PC) \leftarrow (rs1) == (rs2) ? (PC) + sext(offset) : (PC) + 4$

◆ 编码格式:



单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
beq	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst [31 7 30:25 11:8]	RF.rD1	RF.rD2	-	-	-	-

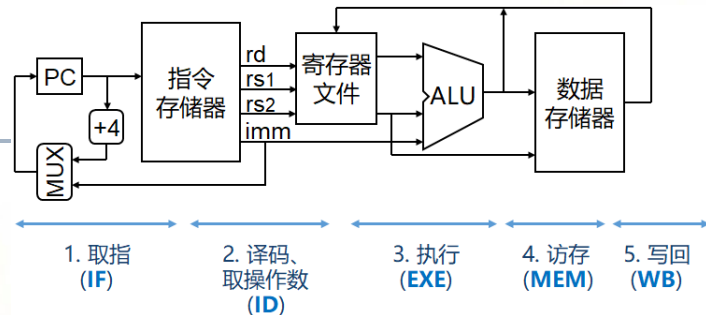
指令级数据通路构建

◆ 示例4: lui指令

◆ U-类型、立即数高位赋值操作

◆ **lui** rd, imm $\# (rd) \leftarrow sext(imm << 12), (PC) \leftarrow (PC) + 4$

◆ 编码格式:



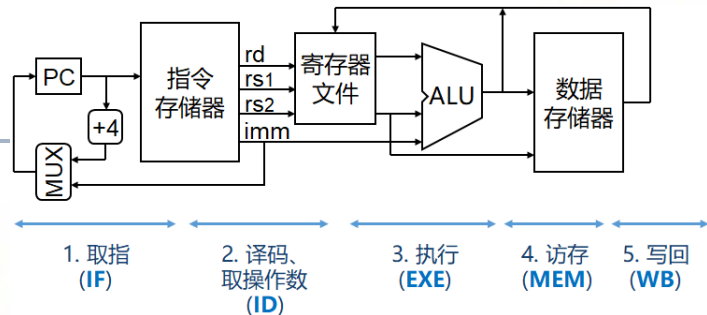
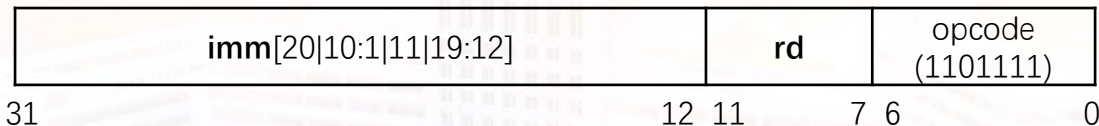
单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
lui	NPC.npc	PC.pc	-	-	-	-	IN.inst[11:7]	SEXT.ext	IN.inst[31:12]	-	-	-	-	-	-

◆ 示例5: jal指令

- ### ◆ J-类型、无条件跳转

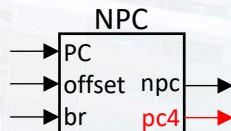
- ◆ **jal rd, offset** $\#(rd) \leftarrow (PC) + 4, (PC) \leftarrow (PC) + sext(offset)$

- ### ◆ 编码格式:



单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ	MEXT		
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
jal	NPC.npc	PC.pc	SEXT.ext	-	-	-	IN.inst[11:7]	NPC.pc4	IN.inst [31 19:12 20 30:21]	-	-	-	-	-	-

- ◆ 修改NPC部件，增加pc4输出信号来实现jal写回：



指令级别数据通路

- ◆ 每一行都是一条指令的数据通路 —— 每行对应的电路能实现一条指令

单元	取指单元				译码单元				执行单元		存储单元				
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
ori	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-
lui	NPC.npc	PC.pc	-	-	-	-	IN.inst[11:7]	SEXT.ext	IN.inst[31:12]	-	-	-	-	-	-
lw	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	MEXT.ext	IN.inst[31:20]	RF.rD1	SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]
beq	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst [31 7 30:25 11:8]	RF.rD1	RF.rD2	-	-	-	-
jal	NPC.npc	PC.pc	SEXT.ext	-	-	-	IN.inst[11:7]	NPC.pc4	IN.inst [31 19:12 20 30:21]	-	-	-	-	-	-

- ◆ 把所有行综合起来，就能得到实现所有指令的数据通路

综合数据通路

◆ Step1: 把每一列的所有情况摘抄到新行

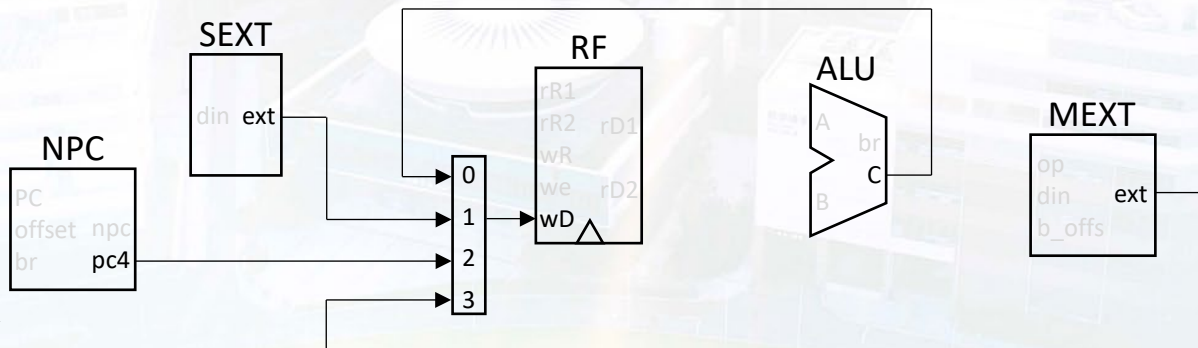
单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
ori	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-
lui	NPC.npc	PC.pc	-	-	-	-	IN.inst[11:7]	SEXT.ext	IN.inst[31:12]	-	-	-	-	-	-
lw	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	MEXT.ext	IN.inst[31:20]	RF.rD1	SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]
beq	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst [31 7 30:25 11:8]	RF.rD1	RF.rD2	-	-	-	-
jal	NPC.npc	PC.pc	SEXT.ext	-	-	-	IN.inst[11:7]	NPC.pc4	IN.inst [31 19:12 20 30:21]	-	-	-	-	-	-
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	IN.inst[11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst[31:20] IN.inst[31:12] IN.inst [31 7 30:25 11:8] IN.inst [31 19:12 20 30:21]	RF.rD1	RF.rD2 SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]

综合数据通路

◆ Step2: 处理多输入情形

单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	IN.inst[11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst[31:20] IN.inst[31:12] IN.inst [31 7 30:25 11:8] IN.inst [31 19:12 20 30:21]	RF.rD1	RF.rD2 SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]

◆ 方法1: 多路选择器



综合数据通路

◆ Step2: 处理多输入情形

单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	IN.inst[11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst[31:20] IN.inst[31:12] IN.inst [31 7 30:25 11:8] IN.inst [31 19:12 20 30:21]	RF.rD1	RF.rD2 SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]

◆ 方法1: 多路选择器



- ① 存在冗余连接, 浪费线网资源
- ② 不够简洁美观

综合数据通路

◆ Step2: 处理多输入情形

单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	IN.inst[11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst[31:7]	RF.rD1	RF.rD2 SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]

◆ 方法2: 拓宽输入 / 增加输入位宽



构造控制逻辑

- ◆ 控制逻辑：根据指令的操作码 (opcode)和功能码 (funct3/funct7), 产生执行指令所需的**控制信号**

- ◆ 控制信号
 - 操作选择信号 指令执行时, 选择部件要完成的具体操作
多功能部件 (NPC、SEXT、ALU、RF等)
 - 多路选择信号 对多输入源进行选择
多路选择器

添加控制信号

◆ 在数据通路表中新增控制信号：

单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
ori	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-
lui	NPC.npc	PC.pc	-	-	-	-	IN.inst[11:7]	SEXT.ext	IN.inst[31:12]	-	-	-	-	-	-
lw	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	MEXT.ext	IN.inst[31:20]	RF.rD1	SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]
beq	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst [31 7 30:25 11:8]	RF.rD1	RF.rD2	-	-	-	-
jal	NPC.npc	PC.pc	SEXT.ext	-	-	-	IN.inst[11:7]	NPC.pc4	IN.inst [31 19:12 20 30:21]	-	-	-	-	-	-
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	IN.inst[11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst[31:7]	RF.rD1	RF.rD2 SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]
操作选择	-	npc_op			-	-	rf_we		sext_op	alu_op		ram_rop		ram_rop	
多路选择	-	-	-	-	-	-	-	rf_wsel	-	-	alub_sel	-	-	-	-

确定控制信号取值

- ◆ 建立控制信号取值表：把数据通路表出现的控制信号加入下表
 - ◆ 结合数据通路表分析每条指令，确定控制信号取值：

指令	opcode	funct3	funct7	npc_op	rf_we	rf_wsel	sext_op	alu_op	alub_sel	ram_rop
ori	0010011	110	-	00	1	00	00	00	00	0
lui	0110011	-	-	00	1	01	01	-	-	0
lw	0000011	010	-	00	1	10	00	01	00	1
beq	1100011	000	-	01	0	-	10	10	01	0
jal	1101111	-	-	10	1	11	11	-	-	0

部件	NPC		
输入	PC	offset	br
ori	PC.pc	-	-
lui	PC.pc	-	-
lw	PC.pc	-	-
beq	PC.pc	SEXT.ext	ALU.br
jal	PC.pc	SEXT.ext	-
操作选择	npc_op		

确定控制信号取值

◆ 建立控制信号取值表：

◆ 结合数据通路表分析每条指令，确定控制信号取值：

指令	opcode	funct3	funct7	npc_op	rf_we	rf_wsel	sext_op	alu_op	alub_sel	ram_rop
ori	0010011	110	-	NPC_PC4	1	WB_ALU	EXT_I	OR	ALUB_EXT	0
lui	0110011	-	-	NPC_PC4	1	WB_EXT	EXT_U	-	-	0
lw	0000011	010	-	NPC_PC4	1	WB_RAM	EXT_I	ADD	ALUB_EXT	WORD
beq	1100011	000	-	NPC_BRCH	0	-	EXT_B	EQ	ALUB_RS2	0
jal	1101111	-	-	NPC_JMP	1	WB_PC4	EXT_J	-	-	0

```
`define NPC_PC4    2'b00  
`define NPC_BRCH  2'b01  
`define NPC_JMP   2'b10
```

NPC_PC4 : NPC部件输出PC+4作为下一条指令地址

NPC_BRCH: NPC部件根据ALU标志位确定下一条指令地址

NPC_JMP : NPC部件输出PC+offset作为下一条指令地址

提取功能部件

◆ 数据通路表的起点：7个基本功能部件

单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
ori	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-
lui	NPC.npc	PC.pc	-	-	-	-	IN.inst[11:7]	SEXT.ext	IN.inst[31:12]	-	-	-	-	-	-
lw	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	MEXT.ext	IN.inst[31:20]	RF.rD1	SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]
beq	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst [31 7 30:25 11:8]	RF.rD1	RF.rD2	-	-	-	-
jal	NPC.npc	PC.pc	SEXT.ext	-	-	-	IN.inst[11:7]	NPC.pc4	IN.inst [31 19:12 20 30:21]	-	-	-	-	-	-
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	IN.inst[11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst[31:7]	RF.rD1	RF.rD2 SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]
操作选择	-	npc_op			-	-	rf_we		sext_op	alu_op		ram_rop		ram_rop	
多路选择	-	-	-	-	-	-	-	rf_wsel	-	-	alub_sel	-	-	-	-

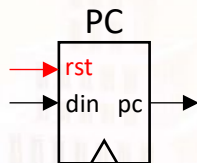
◆ 数据通路已设计好，功能部件的数量、接口等也相继确定

功能部件清单

◆ 取指单元

① PC

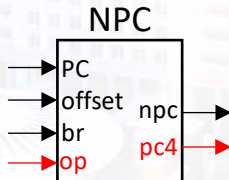
- ✓ 增加复位、时钟



部件	信号名	属性	释义
PC	din	输入	下一条指令地址
	pc	输出	当前指令地址
	clk	输入	时钟信号
	rst	输入	复位信号

② NPC

- ✓ 增加PC+4输出 (jal写回)
- ✓ 增加操作选择信号



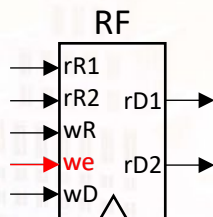
NPC	PC	输入	当前指令地址
	offset	输入	跳转的偏移量
	br	输入	条件分支是否跳转
	npc	输出	下一条指令地址
	pc4	输出	PC+4的值
	op	输入	控制npc的产生

功能部件清单

◆ 译码单元

① RF

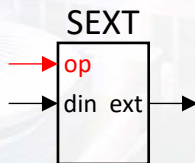
- ✓ 增加时钟
- ✓ 增加操作选择信号(写使能)



部件	信号名	属性	释义
RF	rR1	输入	读地址1
	rD1	输出	读寄存器数据1
	rR2	输入	读地址2
	rD2	输出	读寄存器数据2
	wR	输入	写地址
	we	输入	写使能
	wD	输入	写寄存器数据
	clk	输入	时钟信号

② SEXT

- ✓ 增加操作选择信号



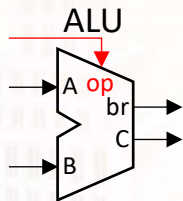
SEXT	din	输入	指令中的立即数
	ext	输出	扩展后的立即数
	op	输入	控制扩展方式

功能部件清单

◆ 执行单元

① ALU

✓ 增加操作选择信号



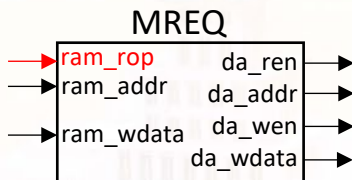
部件	信号名	属性	释义
ALU	A	输入	操作数1
	B	输入	操作数2
	C	输出	运算结果
	f	输出	标志位
	op	输入	控制运算

功能部件清单

◆ 存储单元

① MREQ

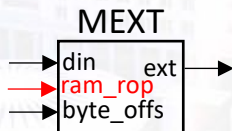
✓ 增加操作选择信号



部件	信号	属性	释义
MREQ	ram_addr	输入	计算得出的访存地址
	ram_rop	输入	控制读访存操作
	da_ren	输出	读请求信号
	da_addr	输出	读地址信号
	ram_wdata	输入	待写入主存的数据
	da_wen	输出	写请求信号
	da_wdata	输出	当前指令地址

② MEXT

✓ 增加操作选择信号



MEXT	ram_rop	输入	控制读访存操作
	din	输入	主存读数据
	byte_offs	输入	访存地址的字节偏移
	ext	输出	扩展后的数据

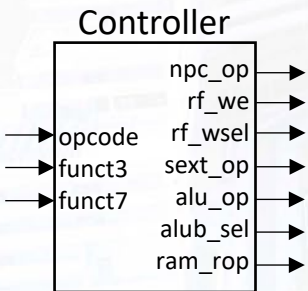
功能部件清单

◆ 控制器

- 输出：数据通路表的操作选择信号、多路选择信号

操作选择	-	npc_op			-	-	rf_we		sext_op		alu_op		ram_rop		ram_rop	
多路选择	-	-	-	-	-	-	-	rf_wsel	-	-	alub_sel	-	-	-	-	-

- 输入：指令的操作码、功能码



部件	信号名	属性	释义
Controller	opcode	输入	指令的操作码
	funct3	输入	指令的3位功能码
	funct7	输入	指令的7位功能码
	npc_op	输出	控制NPC部件产生指令地址
	rf_we	输出	控制寄存器堆的读写操作
	rf_wsel	输出	控制寄存器堆写回数据的来源
	sext_op	输出	控制SEXT部件的符号扩展方式
	alu_op	输出	控制ALU的运算类型
	alub_sel	输出	控制ALU输入B的数据来源
	ram_rop	输出	控制读访存操作

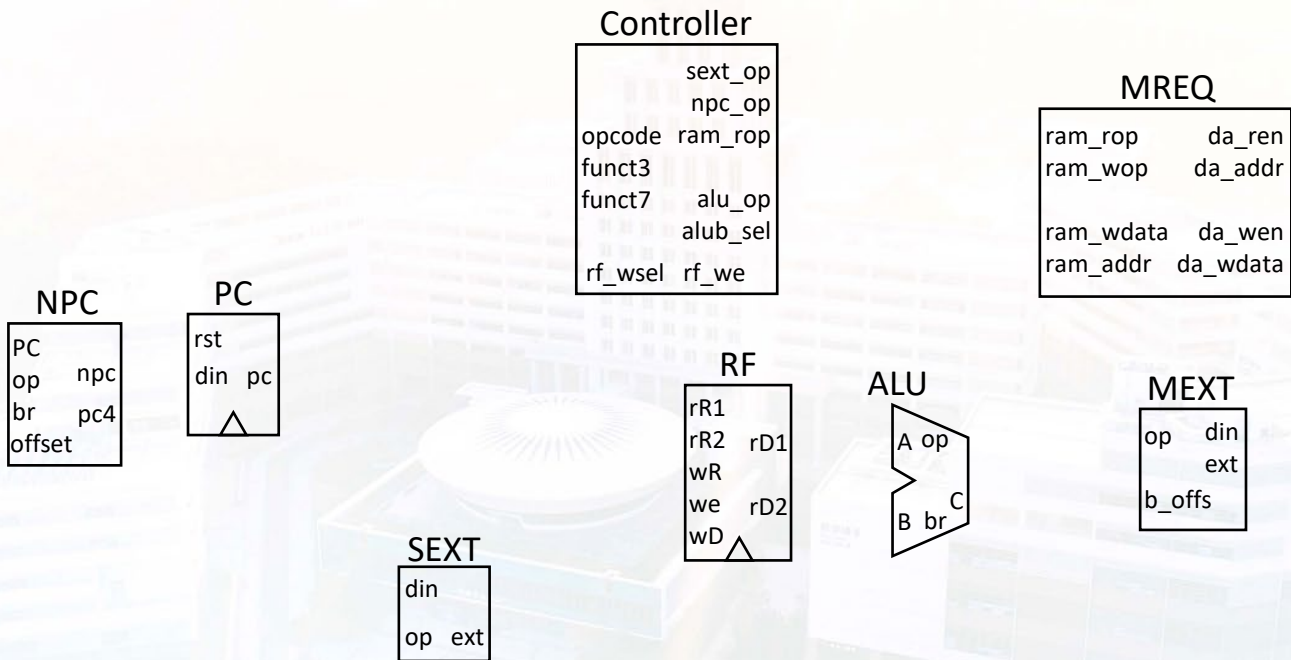
数据通路绘制

- ◆ CPU = 数据通路 + 控制器
 - ◆ 数据通路 = 功能部件 + 功能部件之间的连接关系
 - ◆ 数据通路表描述了功能部件及其连接关系、控制信号
 - ◆ 控制信号取值表描述了控制信号的数字逻辑

单元	取指单元				译码单元					执行单元		存储单元			
部件	PC	NPC			RF				SEXT	ALU		MREQ		MEXT	
输入	npc	PC	offset	br	rR1	rR2	wR	wD	imm	A	B	addr	wd	din	b_offs
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	IN.inst[11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst[31:7]	RF.rD1	RF.rD2 SEXT.ext	ALU.C	-	IN.rd	ALU.C[1:0]
操作选择	-	npc_op			-	-	rf_we		sext_op	alu_op		ram_rop		ram_rop	
多路选择	-	-	-	-	-	-	-	rf_wsel	-	-	alub_sel	-	-	-	-

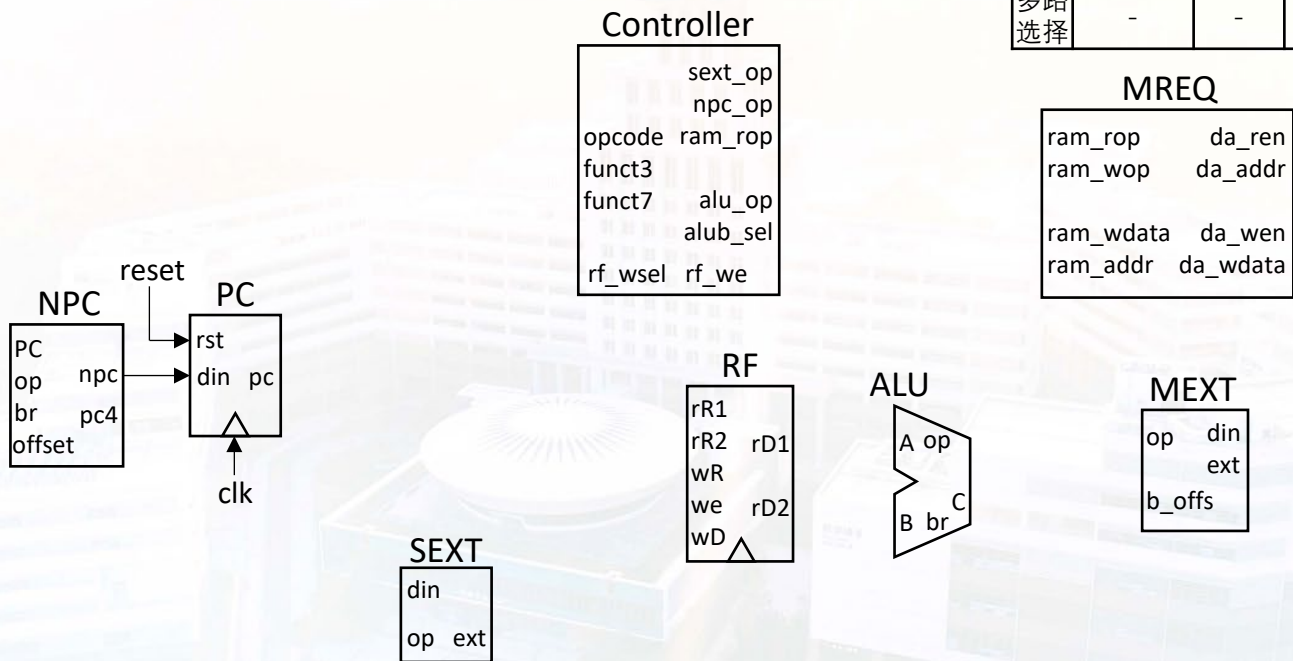
数据通路绘制

◆ 排列所有功能部件：



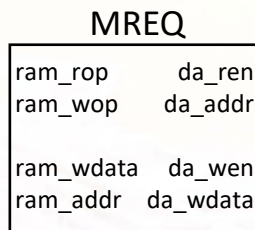
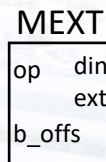
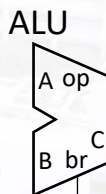
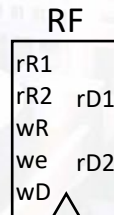
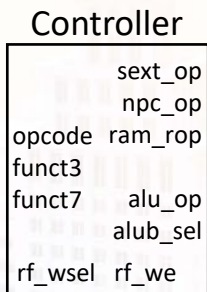
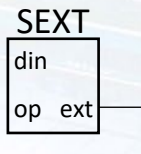
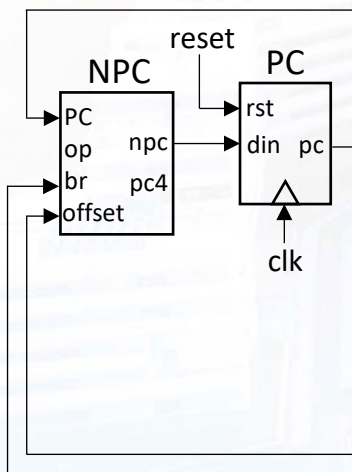
数据通路绘制

单元	取指单元			
部件	PC	NPC		
输入	npc	PC	offset	br
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br
操作选择	-	npc_op		
多路选择	-	-	-	-



数据通路绘制

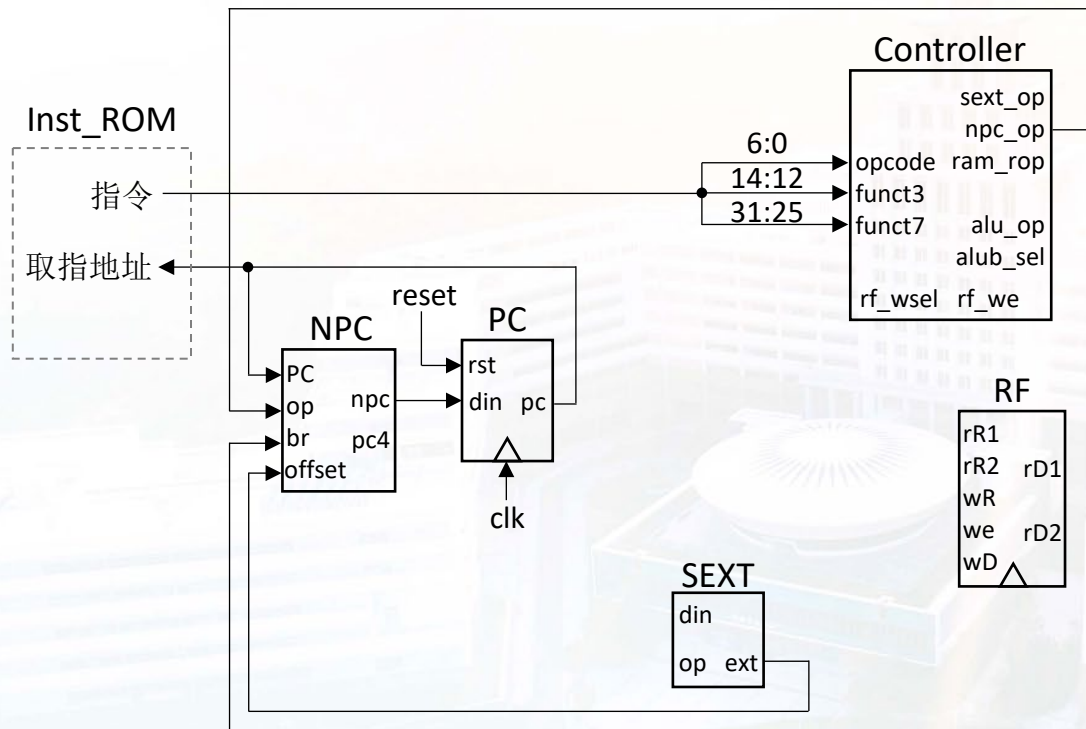
◆ 连接取指单元：



单元	取指单元			
部件	PC	NPC		
输入	npc	PC	offset	br
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br
操作选择	-	npc_op		
多路选择	-	-	-	-

数据通路绘制

◆ 连接取指单元：



单元	取指单元			
部件	PC	NPC		
输入	npc	PC	offset	br
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br
操作选择	-	npc_op		
多路选择	-	-	-	-

MREQ

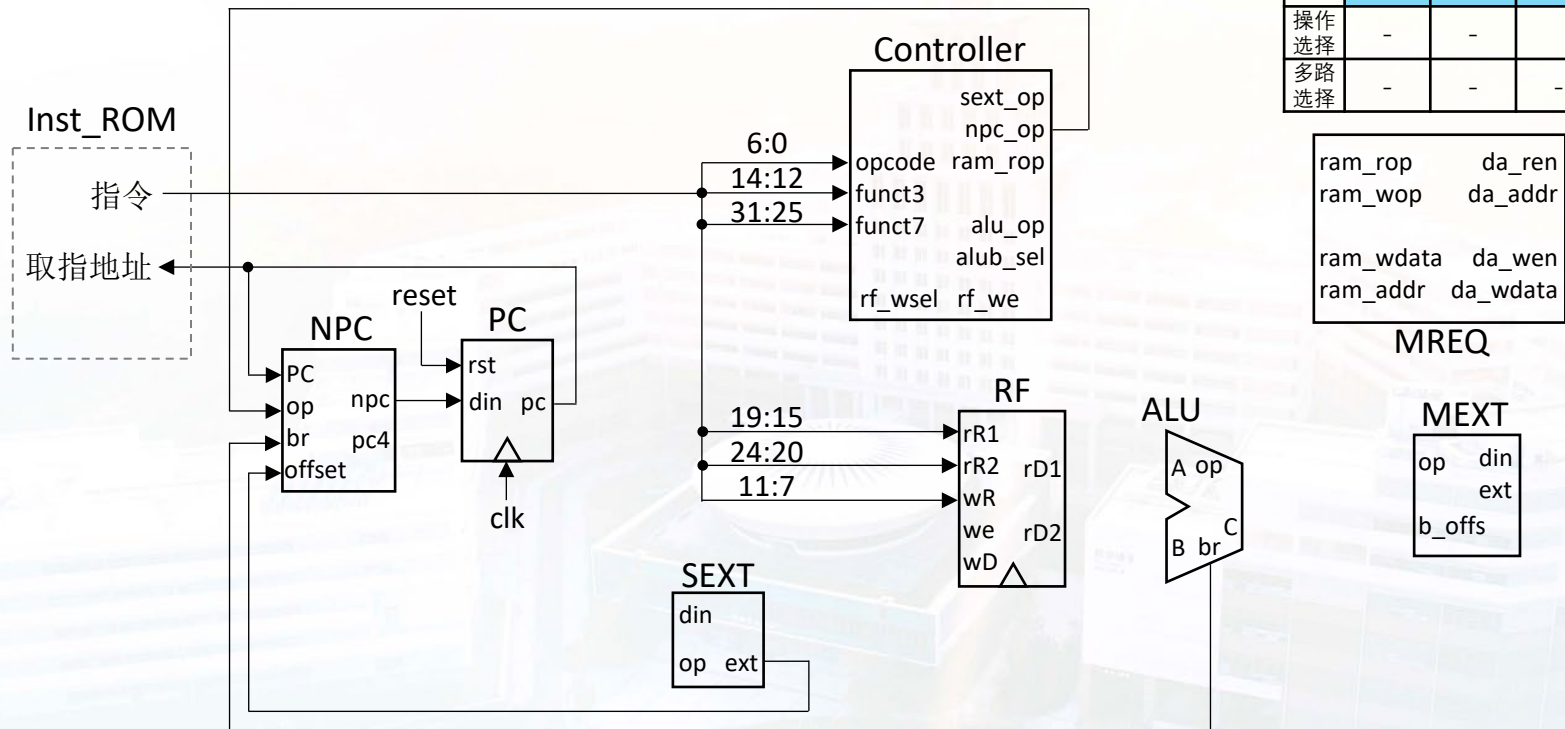
ram_rop	da_ren
ram_wop	da_addr
ram_wdata	da_wen
ram_addr	da_wdata

MEXT

op	din
	ext
b_offs	

数据通路绘制

◆ 连接译码单元：



单元	译码单元				
部件	RF				SEXT
输入	rR1	rR2	wR	wD	imm
完整	IN.inst [19:15]	IN.inst [24:20]	IN.inst [11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst [31:7]
操作选择	-	-	rf_we		sext_op
多路选择	-	-	-	rf_wsel	-

ram_rop	da_ren
ram_wop	da_addr
ram_wdata	da_wen
ram_addr	da_wdata

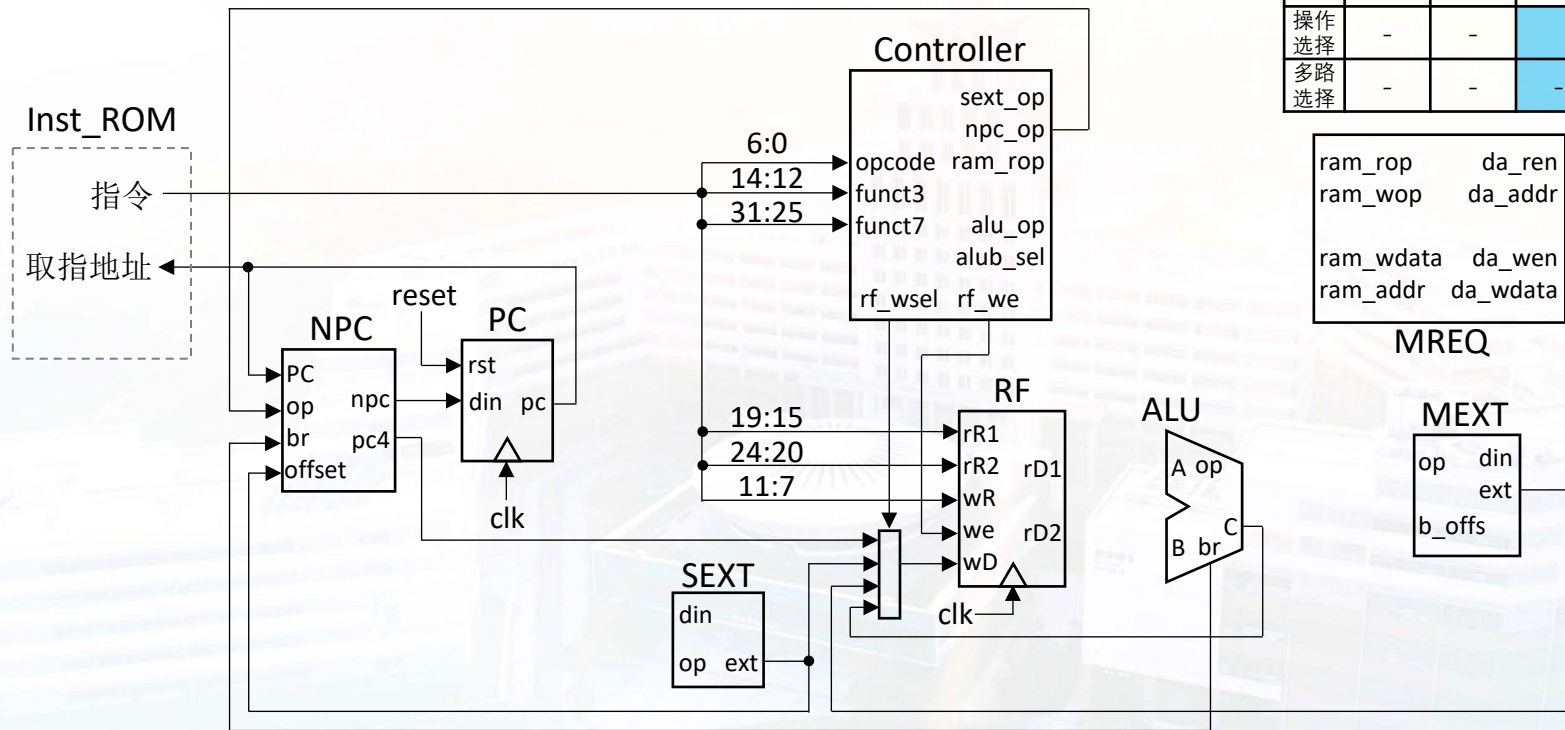
MREQ

MEXT

op	din
ext	
b_offs	

数据通路绘制

◆ 连接译码单元：



单元	译码单元				
部件	RF				SEXT
输入	rR1	rR2	wR	wD	imm
完整	IN.inst [19:15]	IN.inst [24:20]	IN.inst [11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst [31:7]
操作选择	-	-	rf_we		sext_op
多路选择	-	-	-	rf_wsel	-

ram_rop	da_ren
ram_wop	da_addr
ram_wdata	da_wen
ram_addr	da_wdata

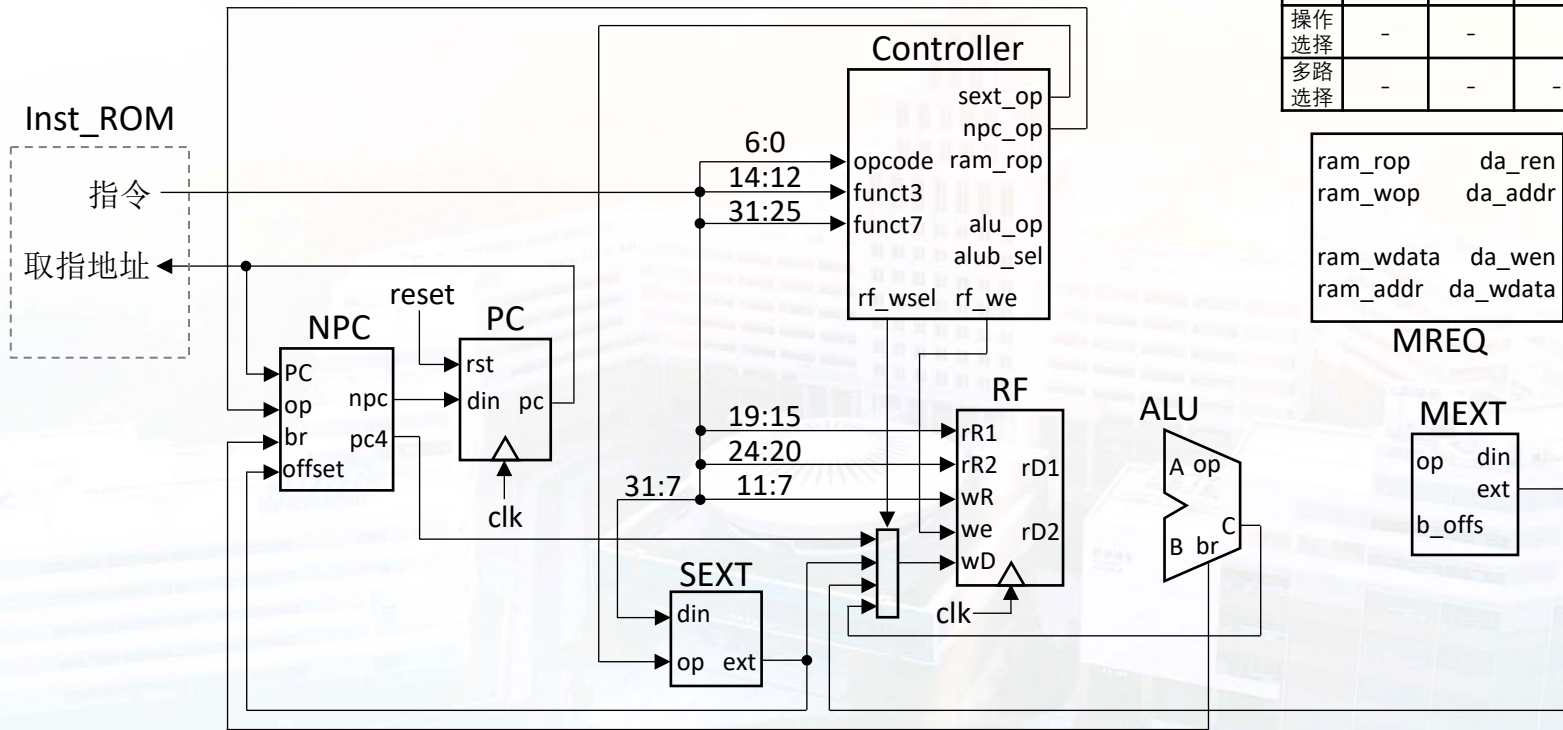
MREQ

MEXT

op	din
ext	
b_offs	

数据通路绘制

◆ 连接译码单元:



单元	译码单元				
部件	RF				SEXT
输入	rR1	rR2	wR	wD	imm
完整	IN.inst [19:15]	IN.inst [24:20]	IN.inst [11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst [31:7]
操作选择	-	-	rf_we		sxt_op
多路选择	-	-	-	rf_wsel	-

ram_rop	da_ren
ram_wop	da_addr
ram_wdata	da_wen
ram_addr	da_wdata

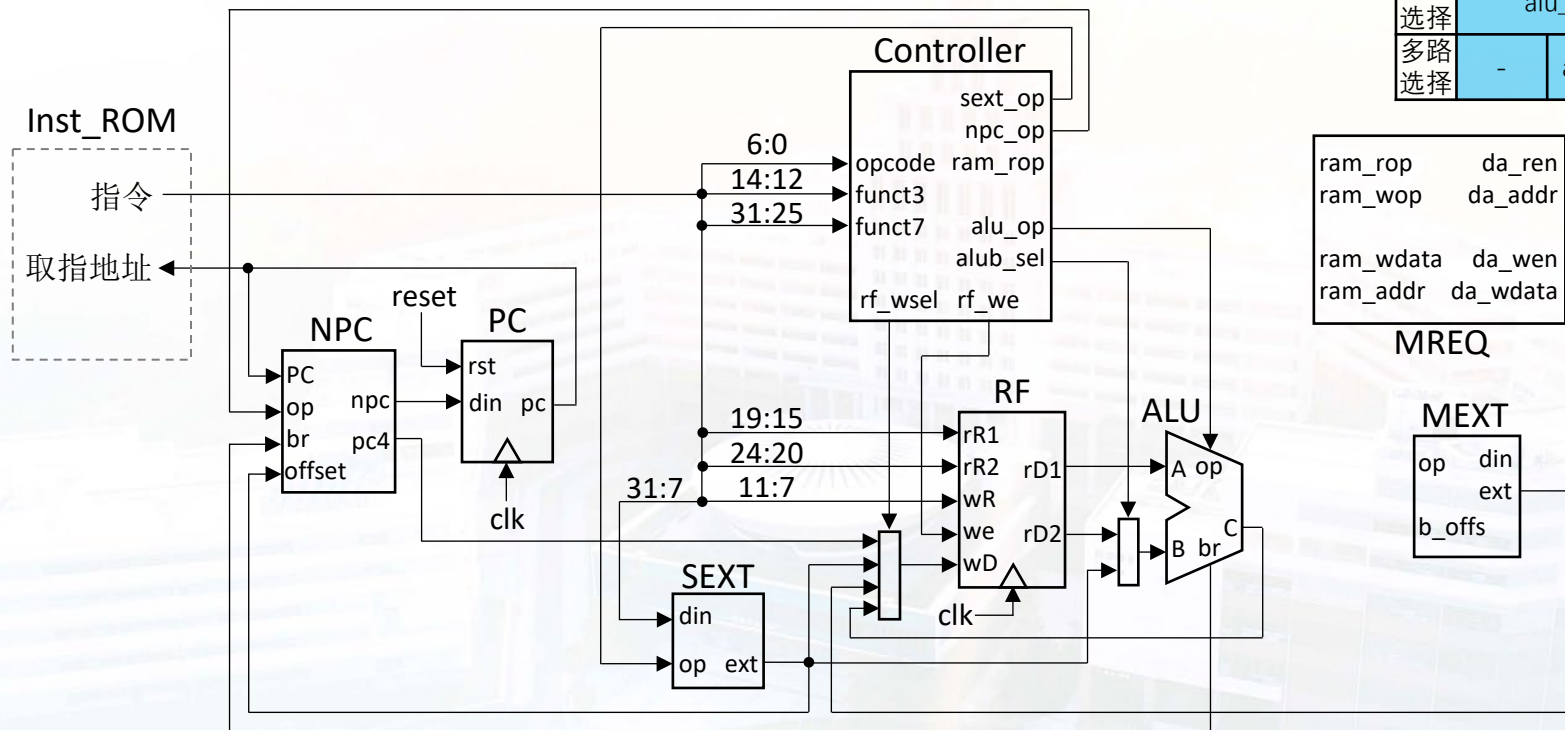
MREQ

MEXT

op	din
ext	b_offs

数据通路绘制

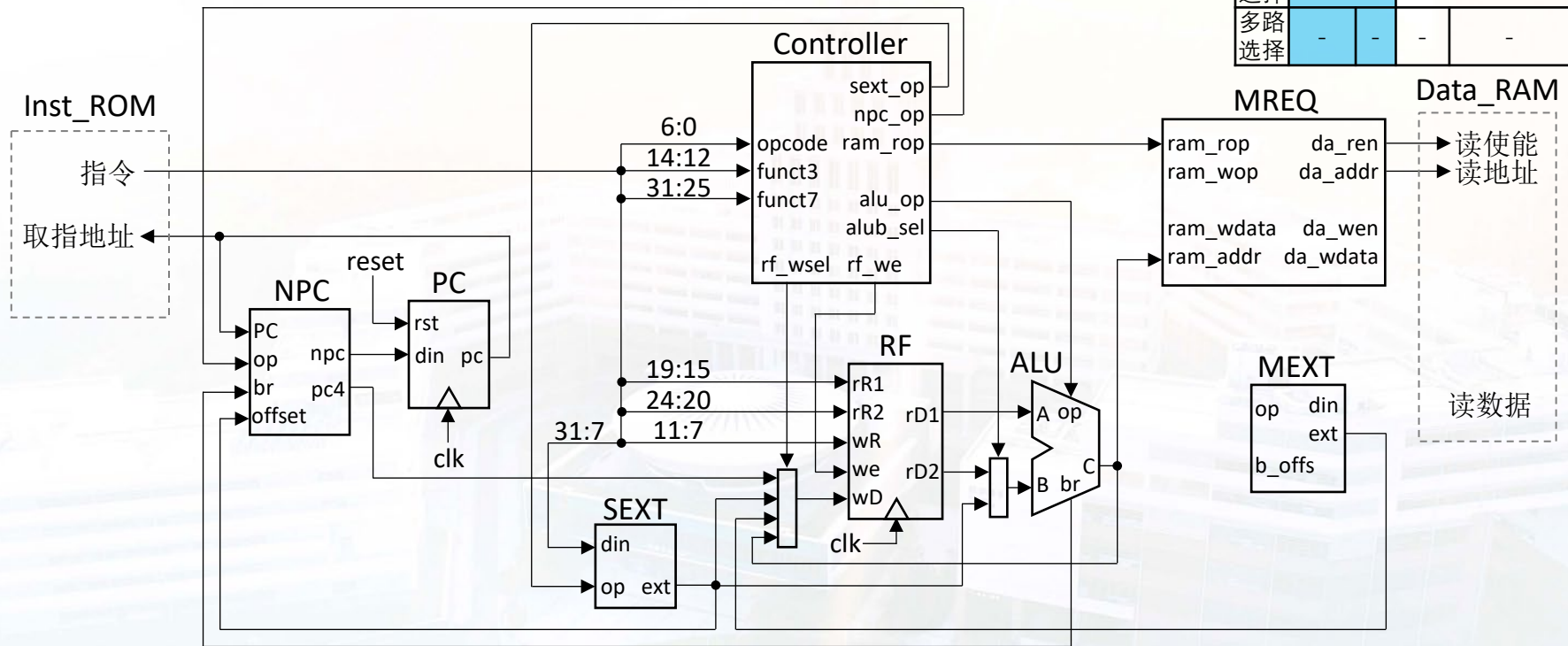
◆ 连接执行单元：



单元	执行单元	
部件	ALU	
输入	A	B
完整	RF.rD1	RF.rD2 SEXT.ext
操作选择	alu_op	
多路选择	-	alub_sel

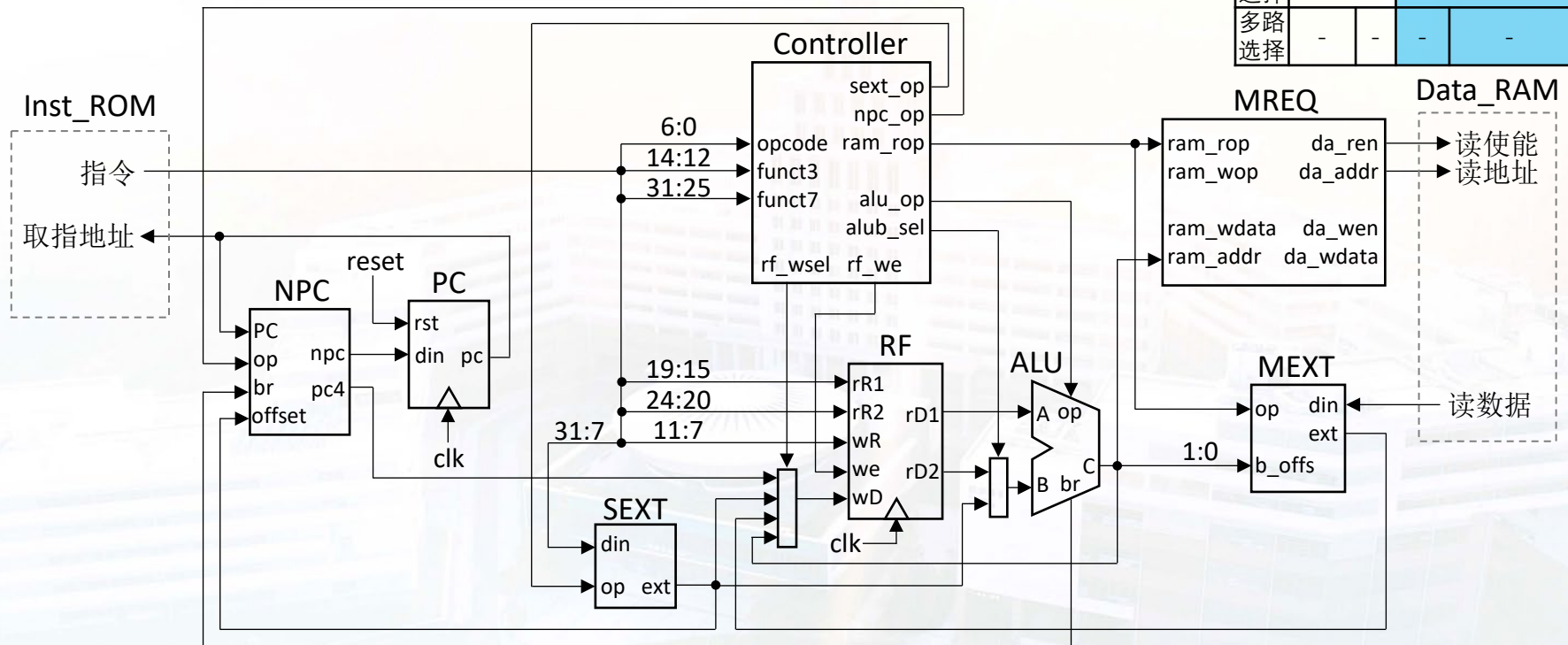
数据通路绘制

◆ 连接存储单元：



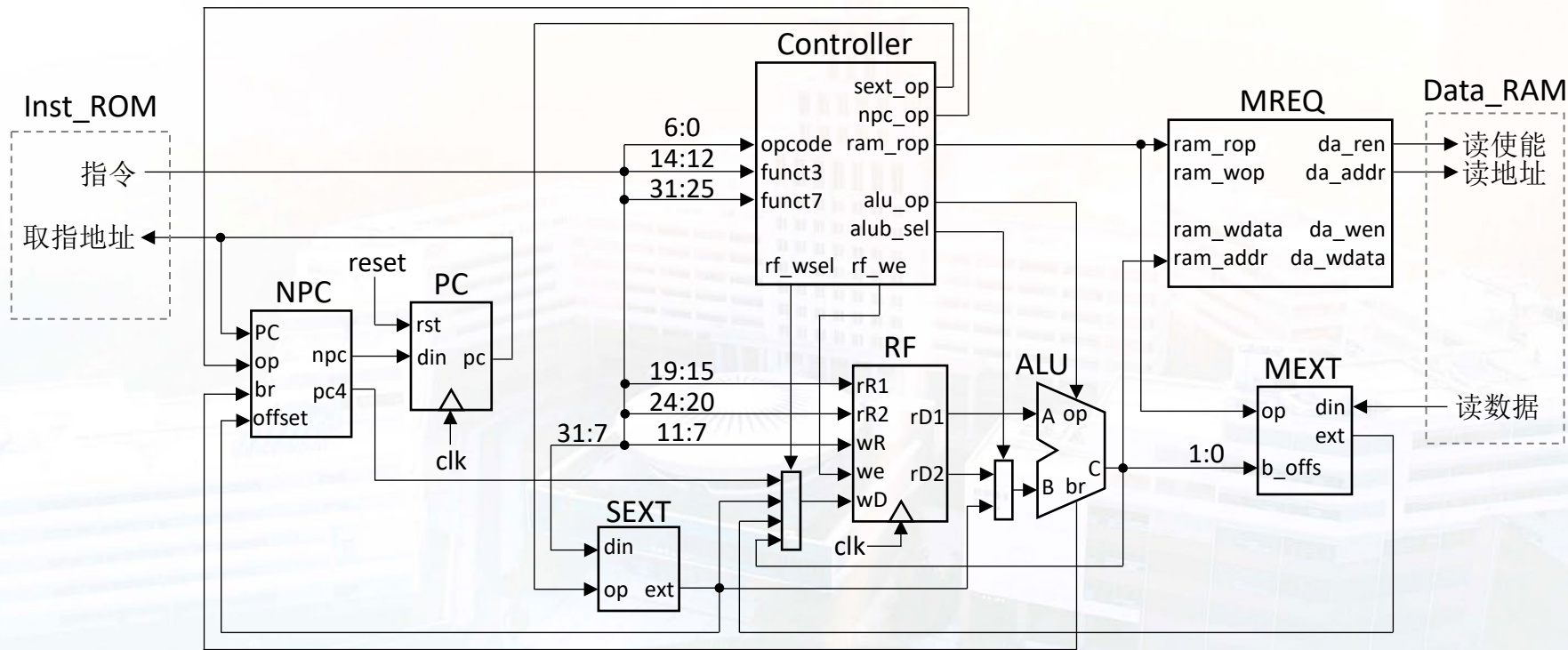
数据通路绘制

◆ 连接存储单元：



数据通路绘制

◆ 完整数据通路示例（5条指令）：



目录

课程介绍

指令系统概要

CPU的功能、原理与结构

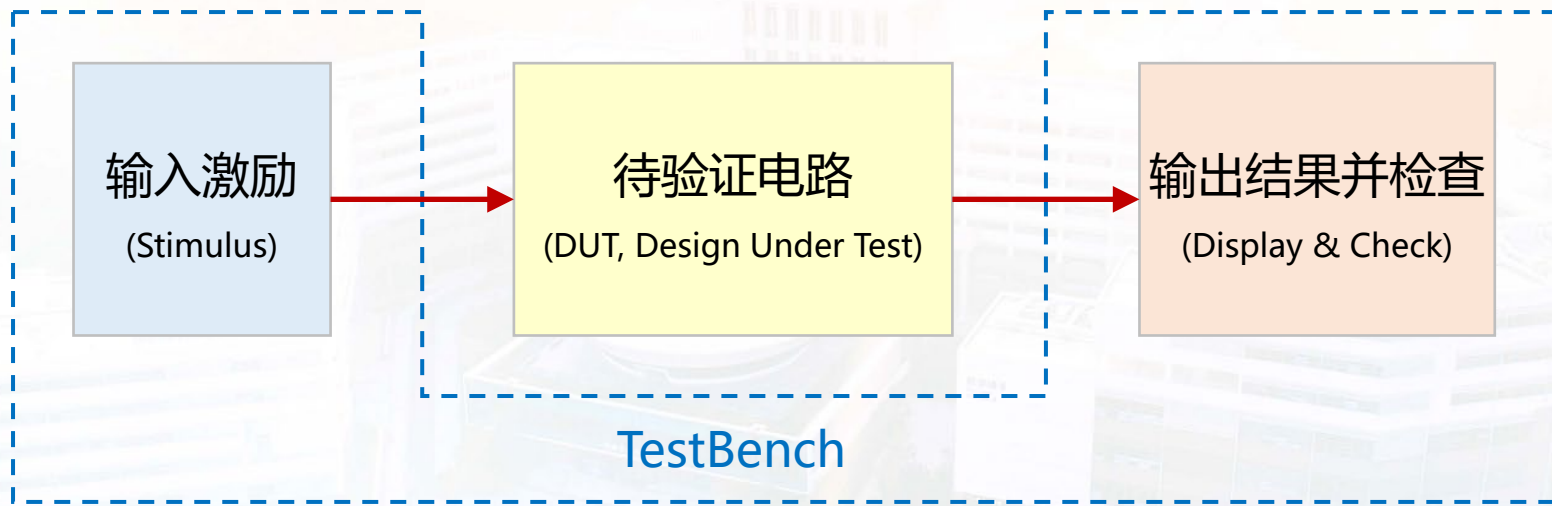
单周期CPU设计

工程化设计方法

CPU功能验证

数字电路的功能验证

- ◆ 如何对数字电路的功能进行验证？ —— 时序仿真
- ◆ 仿真 —— 基于软件模拟 (而非电路实测) 来验证电路功能的方法

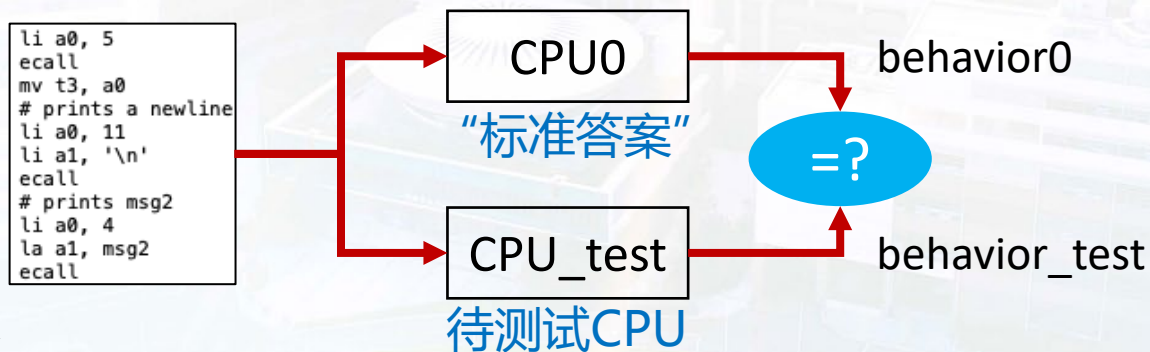


CPU的功能验证

- ◆ CPU也是数字电路，也可采用相同的验证方法，但效率太低
 - ◆ 只能分模块验证，或借助外设输出
 - ◆ 出错点通过测试程序的逻辑路径传递很远之后才能被发现
- ◆ 改进：
 - ◆ 用**一段指令序列**作为输入激励，通过**观察程序执行结果**来验证CPU功能
 - ◆ 验证效率大幅提高，但难以定位错误
- ◆ 更好的方法：基于Trace比对的功能验证

CPU的功能验证 —— 基于Trace比对

- ◆ Trace: CPU执行指令序列时产生的信息 (PC、写寄存器的信息, etc)
- ◆ 基于Trace比对的验证方法:
 - ① 用已知功能正确的CPU运行测试程序, 记录Trace0 (Golden Trace)
 - ② 用待验证CPU运行测试程序, 产生Trace1
 - ③ 将Trace1与Trace0进行实时比对, 如果出现不同, 立即报错并停止



作业 (DDL: 7月19日 23:59) 每个人都要交, 尽早完成!

◆ 完成数据通路和控制信号的构造、综合

所属单元	取指单元				译码单元				执行单元		存储单元				
部件	PC	NPC			RF				ALU		MREQ		MEXT		
输入信号	npc	pc	offset	br	rR1	rR2	wR	wD	imm	A	B	ram_addr	ram_wdata	din	byte_offs
addi	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-
ori	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-
slli	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	ALU.C	IN.inst[31:20]	RF.rD1	SEXT.ext	-	-	-	-
lui	NPC.npc	PC.pc	-	-	-	-	IN.inst[11:7]	SEXT.ext	IN.inst[31:12]	-	-	-	-	-	-
lw	NPC.npc	PC.pc	-	-	IN.inst[19:15]	-	IN.inst[11:7]	MEXT.ext	IN.inst[31:20]	RF.rD1	SEXT.ext	ALU.C	-	IN.daccess_rdata	ALU.C[1:0]
beq	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst[31:7][30:25][11:8]	RF.rD1	RF.rD2	-	-	-	-
bne	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	-	-	IN.inst[31:7][30:25][11:8]	RF.rD1	RF.rD2	-	-	-	-
jal	NPC.npc	PC.pc	SEXT.ext	-	-	-	IN.inst[11:7]	NPC.pc4	IN.inst[31:19:12][20:30:21]	RF.rD1	SEXT.ext	-	-	-	-
完整	NPC.npc	PC.pc	SEXT.ext	ALU.br	IN.inst[19:15]	IN.inst[24:20]	IN.inst[11:7]	ALU.C SEXT.ext MEXT.ext NPC.pc4	IN.inst[31:7]	RF.rD1	RF.rD2 SEXT.ext	ALU.C	-	IN.daccess_rdata	ALU.C[1:0]
操作选择信号	-	npc_op			-	-	rf_we		sext_op	alu_op		ram_rop		ram_rop	
多路选择信号	-	-	-	-	-	-	-	rf_wsel	-	-	alub_sel	-	-	-	-

◆ 完成控制信号的取值表

指令	opcode	funct3	funct7	控制信号						
				npc_op	rf_we	rf_wsel	sext_op	alu_op	alub_sel	ram_rop
addi	0010011	000	-	PC4	1	WB_ALU	I_TYPE	ADD	SEL_EXT	0
ori	0010011	110	-	PC4	1	WB_ALU	I_TYPE	OR	SEL_EXT	0
slli	0010011	001	0000000	PC4	1	WB_ALU	I_TYPE	SLL	SEL_EXT	0
lui	0110111	-	-	PC4	1	WB_EXT	U_TYPE	-	-	0
lw	0000011	010	-	PC4	1	WB_RAM	I_TYPE	ADD	SEL_EXT	WORD_EXT
beq	1100011	000	-	BRANCH	0	-	B_TYPE	EQ	SEL_RS2	0
bne	1100011	001	-	BRANCH	0	-	B_TYPE	NE	SEL_RS2	0
jal	1101111	-	-	JUMP	0	WB_PC4	J_TYPE	-	SEL_EXT	0



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ